

Mobile Code Security

Module 21

Duration: 26:38

Video: <https://www.youtube.com/watch?v=ccqp2LU6E7Q>

Course Coordinator - Dr Maninder Singh, Professor and Dean, Academic Affairs, Thapar University, Patiala

Subject Expert - Dr Navdeep Singh, Department of Computer Engineering, Punjabi University, Patiala

Auto-generated YouTube transcript for personal study. Spoken wording belongs to the original lecture. Follow the video link above; this file is not an official CEC/SWAYAM publication.

my dear students hello I welcome you all in our series of lectures on cyber security in the previous lecture we studied about Information Security Management today we are going to discuss mobile code security and the various issues associated with it so let's start with it the mobile code Paradigm and Compasses programs that can be executed on one or several host other than the one that they originate from mobility of such programs implies some built-in capability for each piece of code to travel smoothly from one host to another a mobile code is associated with at least two parties is producer and its 1 minute, 1 second consumer the consumer being the host that runs the code mobile code systems range from simple applets to intelligent

software agents these systems offer several advantages over the more traditional distributed computing approaches flexibility in software design beyond the well-established object-oriented Paradigm and bandwidth optimization just to name two of them as usual increased flexibility comes with a cost that is increased vulnerability in the face of malicious intrusion scenarios akin to internet possible vulnerabilities with mobile code fall in one of the two categories attacks performed by a mobile program against the remote host on which the program is executed as with malicious applets or ActiveX programs and the less classical category of attacks due to the subversion of the mobile code and its data by the remote execution

environment now let's discuss mobile code security threats a first security threat appears if the mobile code generated by a malicious Outsider attacks the environment where it is executed this problem bears some similarity with Trojan horses but mobile code brings up other issues such as mobile code aims at transparency Automation and a wider scale of execution moreover vulnerabilities of the environment are not the only Targets in addition to this relatively classical scenario mobile code introduces a completely new problem such as the host itself might be the malicious party trying to subvert the mobile when protecting a host from potentially malicious code code Mobility imposes some of the security

features such as number one host and mobile code be of separate identities therefore the mobile code's origin must be authenticated number two mobile code is exposed through the network hence the host must verify the Integrity of the mobile code it just received number

three the host does not generate the mobile code but another party does consequently the actions it performs must be limited through access control and checked through semantic verification when protecting a mobile code from a potentially malicious host code. Mobile implies that the program will be run under total control of the host this means the threats such as spoofing through impersonation of code owner theft and secrecy violation.

through unauthorized disclosure. Integrity violation through subversion of code semantics to prevent all these cases data segments as well as code semantics must be protected. Next is protection of a host from a mobile code. The subject of host protection is very important. The first answer in this area was to simply limit the functionality of the execution environment in order to limit the vulnerabilities. Techniques for protection of host now evolve along two directions: a mobile core infrastructure that is gradually enhanced with authentication, data integrity and access control mechanisms; verification of mobile code semantics. There are various techniques detailing these aspects. Let us discuss them one by one.

Boxing Sand boxing consists in running a mobile code in a restricted environment called The Sandbox. Otherwise untrusted mobile code can be executed without worrying. In the sandbox a sand box can be characterized by two different mechanisms: number one it confines code either through type checking language Properties or the use of protection domains to prevent the subversion of trusted code and number two it enforces a fixed policy for the execution of code. This approach is especially illustrated by the early Java JDK 1.0 where it was used in order to enable applets available anywhere on the internet to run within a browser. The major drawback of sandboxing is due to the fact that applications running in such a restrictive environment are themselves seldom useful.

Next is digital shrink wrap. Another approach to protect an execution environment against potentially malicious mobile code is to authenticate the mobile code before it is actually executed. In this approach the producer of the code is required to sign it and the code consumer verifies the signature of the producer before using it. Although it is not possible to decide whether a given piece of mobile code contains malicious code, one can at least determine whether it is authentically coming from its claimed source. Microsoft has proposed this approach in its Authentic Code technology. Note that the two approaches sandboxing and digital shrink wrap can be combined to provide more sophisticated protection.

Schemes Sun Microsystems has combined both in its new Java 1.2 security model. Number three: proof carrying code. Recently Neula and Lee from K Mellon University propose a new technique to protect a host from a malicious mobile code. The technique is called proof carrying code (PCC). PCC enables a host to determine automatically and with certainty that a program code provided by another system is safe to install and execute. The basic idea of PCC is that the code producer is required to provide an encoding of a proof that his/her code adheres to the security policy specified by the code consumer. The proof is encoded in a form that can be transmitted digitally. Therefore the code consumer can quickly validate the code using a simple

automatic and reliable proof checking process next is steps for generating and verifying the proof carrying code also called PCC a typical PCC session requires five steps to generate and verify the PCC step number one apcc session starts with the code producer preparing the untrusted code to be sent to the code consumer the producer adds annotation to the code which can be done manually or automatically by a tool such as a certifying compiler these annotations contain information that helps the code consumer to understand the safety relevant properties of the code the code producer then sends the annotated code to the code consumer to execute it step number two the code consumer performs a fast but detailed inspection

of the annotated code this is accomplished using a program called VC gen which is one of the component of the consumer defined safety policy VC gen performs two tasks first it checks simple safety properties of the code for example it verifies that all immediate jumps are within the code segment boundaries second VC gen watches for instructions whose execution might violate the safety policy when such an instruction is encountered VC gen emits a predicate that expresses the conditions under which the execution of the instruction is safe the collection of the verification conditions together with some control flow information make up the safety predicate and a copy of it is sent to the proof

producer step number three a on receiving the safety predicate the proof producer attempts to prove it and in the event of success it sends an encoding of formal proof back to the code consumer because the code consumer does not have to trust the proof producer any system can act as a proof producer step number four then the code consumer performs a proof validation this phase is performed using a proof checker the proof checker verifies that each inference step in the proof is a valid instance of one of the axioms and inference rules specified as part of the safety policy in addition the proof checker verifies that the proof proves the same safety predicate generated in Step number two step five finally after the executable code has passed both the VC

gen checks and the proof check it is trusted not to violate the safety policy it can thus be safely installed for execution without any further need for runtime checking next is code signing code signing is the process by which a code is digitally signed by the code producer in order to assure strong authentication and integrity of the code to the code consumer this model was first introduced by Microsoft within the active X framework Java jtk 1.1 also follows the code signing model with so-called applets upon receipt of an applet with a valid signature the code consumer Java virtual machine executes the applet like a trusted piece of code authorizing it to access all features available in Java an applet without a proper signature is

run in a sandbox as in the previous version of the jdk there is more to scaring a host from a malicious mobile code than just making sure that this program has been correctly signed by someone on the internet the latter simply does not imply that the code signer must be trusted unrestrictedly moreover the presumption underlying code signing that users can decide

whether to run a program based on a signature has questionable validity a later drawback of code signing lies in the rudimentary form of Access Control provided a sign code is either granted full access to the resources of the code consumer or not executed at all this choice is left to the end user who even without administrator privileges can put the entire security

at risk number five access control the only way to limit this kind of security threat is to enable more complex Access Control schemes in order to limit the impact of an attack this can be seen as the refinement of a monolithic sandbox policy into smaller application specific policies the identity of the signer of a code AS Des described before also helps to further refine the execution policy definition provided that it can be established by some mean like a public key infrastructure for instance Java GDK 1.2 security model follows this scheme and thus permits the definition of finer Grant security policies more suited to executing an untrusted mobile code even more recently Sun releases the Java authentication and authorization services or Jazz which aims at

integrating the identity of the user running the mobile code in the definition of the access control compared with sandboxing and code signing the access control model has the best of both worlds the actions performed by a mobile code can be restricted to some resources while it permits at the same time to write and run really useful software yet the enforcement of the excess scheme has a cost since it is performed dynamically at runtime number six program checking checking a mobile code means to perform a verification on the code structure or on the code behavior as it is run and modifying in consequence the status of the code for instance from trusted to untrusted checking also involves the notion of

verification against a given security policy sandboxes already exercise some form or another of rudimentary program check either statically for instance to ensure that operands of an instruction are of the correct type or dynamically for example to locate any access to a protected resource a newer approach to host protection is to statically check the mobile code the program is then run without any expensive runtime checks this approach is for instance taken in the proof carrying code scheme and even to some extent in Java's virtual machine for safety checks the proof carrying code scheme or PCC is a good instance of the static program checking model in this scheme a predefined security policy is defined in terms of a

logic the host first asks to be sent a proof that the code respects the policy before he actually agrees to run it the code producer then sends the program and an accompanying proof using a set of sound XMS and rewriting rules as defined by the chosen logic and shared by the code producer and the code consumer after receiving the code the host can then check the program with a guide guidance of the proof this can be seen as a form of type checking of the program since the proof is directly derived from it the most interesting part of PCC is that type checking the proof is relatively obvious compared to proving the program and does not impose much computational burden on the code consumer moreover since the proof is not

the program PCC already permits to express complex safety properties it seems promising for security properties as well however automating the proof generation is still a non-trivial problem and to date proofs generally have to be generated by hand next is protection of a mobile code from a malicious host the problem of protection from a malicious host has been studied only recently and is intrinsically more difficult because the environment gets a total control over the mobile code otherwise host protection would not be possible the approaches used for that purpose can be classified along two criteria number one data versus code protection and number two Integrity or confidentiality based data protection is essentially

interested in protecting mobile agents roaming in the internet an agent might encounter a malicious host occasionally who will try to brainwash the agent for instance on the other hand code protection issues address a more systematic form of maliciousness in which the environment where the mobile code runs cannot be trusted now let's learn more about data protection mobile or roaming agents are form of mobile code especially publicized in electronic Commerce but usual data protection techniques are not well adapted for protecting their data mobile agents are used to perform a function on behalf of the user like for instance collect data about a product from several merchants or perform a transaction a good example might be the so-called comparison shopping for

instance an agent looks for the lowest priced book among several retailers in that kind of scenario it is necessary to know whether the data collected have been changed or not if the itinerary fixed for the chain of operations has been followed in an auction scenario it might also be necessary to prevent a host from exploiting the offers made by the previous bidders the Integrity of the data collected by a mobile agent might be protected using a cryptographic technique a first possible technique is simply to digitally sign the result the problem with this approach is that the size of the agent grows linearly as it gathers results a more efficient approach in terms of computational complexity and space is to use hash chaining to achieve

forward Integrity Integrity against deletion or modification of parts of the collector data in different host is achieved until the first malicious host a partial result authentication code is small value ensures the Integrity of all the offers this solution is extended with a cryptographic technique that allows host to update their previous data without increasing the space requirements these techniques have to be enhanced with digital signatures if non repudiation is required on its itinerary from host to host the agent carries an increasing amount of data that might be exploited by a malicious host their confidentiality can be achieved at each step of the agent itinerary by a simple and siring on the current host before the agent moves elsewhere with RSA based encryption the

security will be ensured but the data carried by the agent can be very small the size of the data padding will then be 21 minutes, 1 second excessive sliding encryption and aims at retaining an equivalent security using a large key while at the same time taking into account The Limited storage of an agent sliding encryption is aimed at conserving space rather than

time which might be of importance for agents that collect small amounts of data on many different host next is integrity of computation a mobile code might be rendered for tolerant by performing the work to be done several times this applies in the case of mobile agents the same mobile agent code can be distributed to replicate host and the result is discovered through a vote

Alternately and more convincingly it is also feasible to replicate agents and slightly alter their behavior in order to spoof a malicious host a mobile code might also be protected using a cryptography based Integrity proof or rather a hint of the Integrity of its computation this proof intends to guarantee that the computation was done according to the instructions of the mobile code the trace of the program computation is used as a form of proof showing how the result was obtained in order to make the proof checking process faster this proof is transformed into a holographic proof a holographic proof system is a system for writing and checking proofs in which one can probabilistically check the validity of a proof by examining

only a few of its bits moreover this checking can be performed in poly logarithmic time the transformation of the proof into holographic proof is computationally intensive but it is the host where the agent is executing and not the verifier which performs the computation another problem arises the validity of the holographic proof is only preserved in so far as the bits of the proof carried by the verifier remain unknown to the host that performed the computation this could be ensured if the holographic proof could be sent to the verifier host but a holographic proof is even bigger than the proof it was obtained from this problem is solved by using a cryptographic private information retrieval technique with this technique the

holographic proof remains on the host which computed it and can be queried without the host knowledge of the queried bits the result of these hidden queries and the computation result is finely smaller than the original proof itself then comes privacy of computation a mobile code might be written in order to rely on tamper proof hardware for instance a smart card as a secure kernel for executing secret functions and keeping secret data in that case the achieved services are integrity and partial confidentiality of the program a mobile code might finally be protected algorithmically by hiding the computed function this results in a standalone code without additional Hardware achieving confidentiality of the computation this technique is known as function hiding in the function hiding

scheme If function f is encrypted into EF by its Stander then EF is run on the potentially malicious site with X as an input the result of EFX is returned and decrypted yielding the result of FX computations involving matrices can be hidden and all functions that can be represented by a matrix as well like for example combinatorial Boolean circuits with this our today's lecture is over today we learned that mobile code is generally advocated for bringing substantially increased flexibility and extensibility but these do not come without a price on security as we saw a good level of host protection is achievable there is currently a trend towards enabling finer grain Access Control schemes thanks to this mobile CES will probably

find more useful applications in the near

future semantic based analysis might also provide a new means of securing host without compromising performances on the other hand the protection of a mobile code against a Malaya host is still an open search topic anyhow it can be readily observed that non-cryptographic techniques are generally not sufficient to protect a mobile code thank you and have a nice day